

Experiment Report: Specialized Experiment 4

Flood Inundation Analysis (DEM-based)

Mahmudul Hasan (Student ID: 4125999049)

May 23, 2026

Abstract

Bathtub inundation on a reproducible 100×100 synthetic DEM: mask $z < h$, depth $\max(0, h - z)$, area and volume statistics, GIS-style maps with elevation contours, annotated growth on the flood curve, seed sensitivity, and performance benchmarks. All guide deliverables are present; extensions remain assignment-safe (no routing or PDE solvers).

Key Findings (Executive Summary)

Finding	Result
DEM range (seed 42)	36.0–68.8 m
Flooded @ 40 m / 50 m	2.41% / 25.25%
Flood volume @ 40 m / 50 m	≈ 285 / $\approx 12,278$ m ³
Curve resolution	0.5 m steps, 21 points, monotonic
Seed sensitivity @ 50 m	25.25% / 25.45% / 25.04% (seeds 42/7/99)
Validation checklist	9/9 PASS
Automated tests	21 passed

1 Role in the Integrated Smart Water Pipeline

Experiment 4 is the **impact assessment stage** at the end of the pipeline. Given a water-surface elevation (design flood stage or reservoir spill level), it:

- maps inundation extent, depth, flooded area percentage, and stored flood volume on a reproducible synthetic DEM;
- documents monotonic stage–area–volume behaviour for defensible emergency-response interpretation;
- supports decisions triggered by upstream alerts (Experiment 1), runoff forcing (Experiment 2), and release scenarios (Experiment 3).

It translates hydrologic and operational inputs into spatial flood consequences for planning and communication.

2 Part 1: DEM Data Preparation (15%)

2.1 Synthetic terrain

Grid: 100×100, `rng(42)`, central valley + hills + ridge + noise, clipped 30–80 m. Cell size 1 m. `dem_data.npy` saved by `load_dem()`.

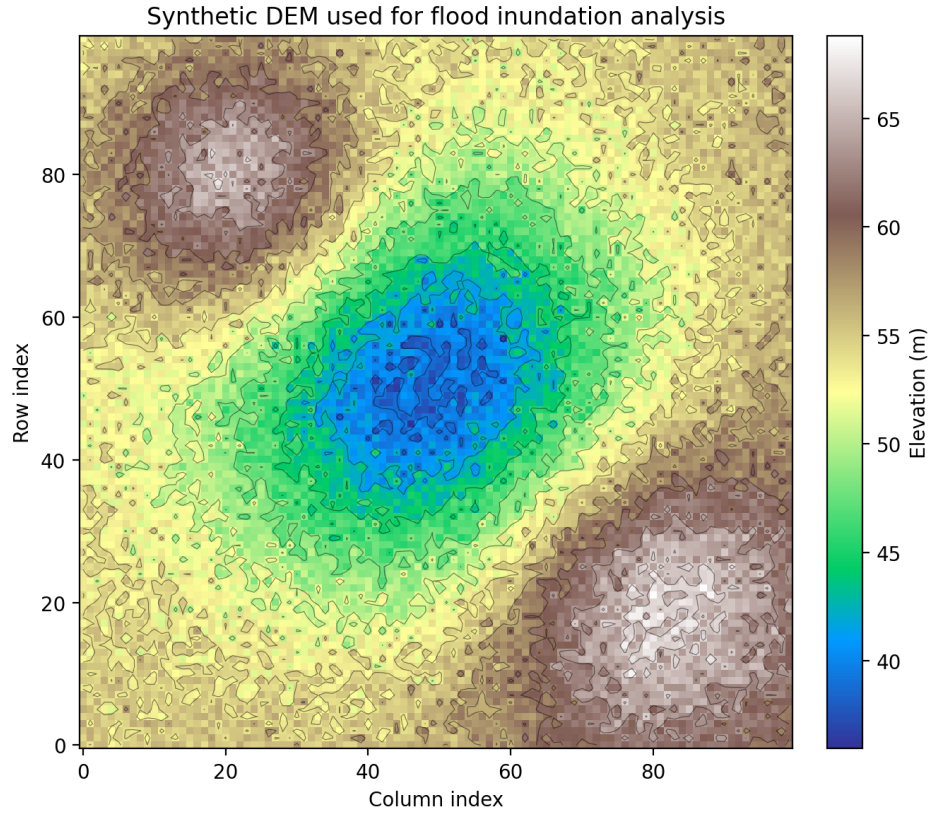


Figure 1: Synthetic DEM with terrain colormap and elevation contours.

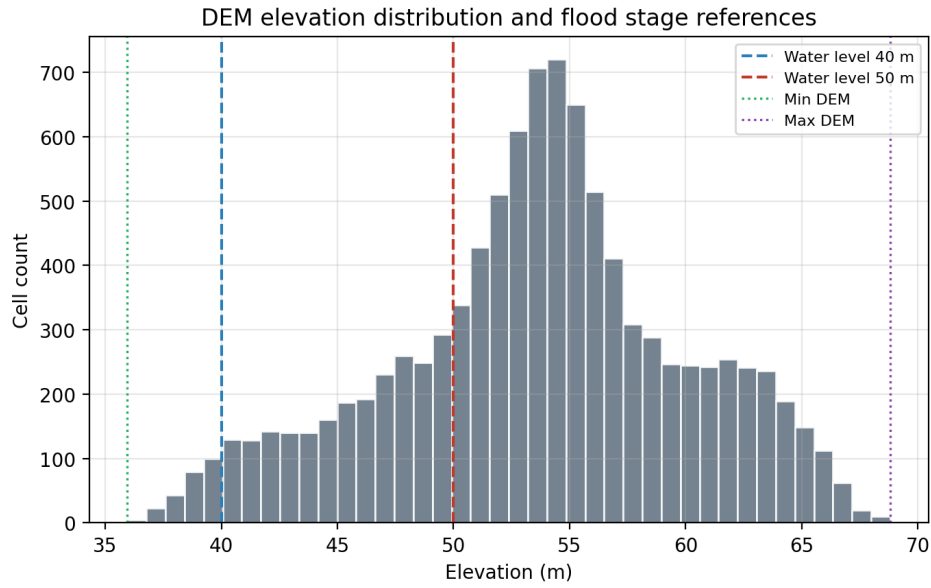


Figure 2: Elevation histogram: vertical lines at 40 m and 50 m stage explain rapid inundation growth once h crosses dense elevation bins.

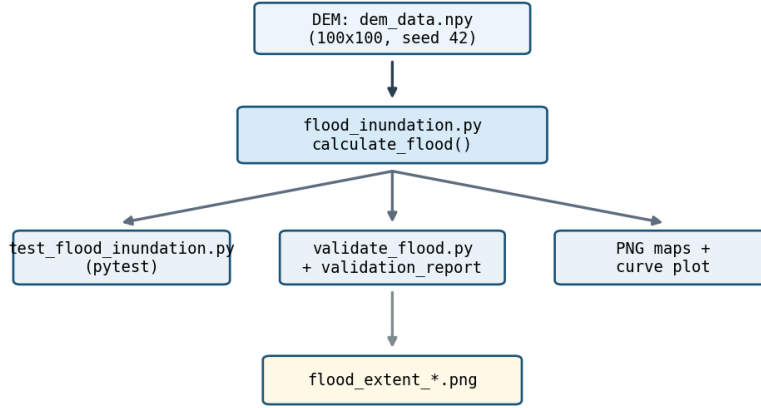


Figure 3: Pipeline: DEM $\rightarrow$ `calculate_flood` $\rightarrow$ visualization $\rightarrow$ validation.

3 Part 2: Flood Calculation (25%)

Mask: $z_{ij} < h$. **Depth:** $\max(0, h - z)$ on wet cells. **Volume:** $V = \sum d_{ij} A_{\text{cell}}$.

Table 1: Statistics at required levels (seed 42 DEM).

h (m)	Flooded (%)	Mean depth wet (m)	Volume (m ³)
40	2.41	1.18	≈ 285
45	10.92	—	—
50	25.25	4.86	$\approx 12,278$

4 Part 3: Visualization (25%)

Maps: grayscale DEM + black elevation contours + blue depth overlay (`origin='lower'`, 300 DPI). Required: `flood_extent_40m.png`, `flood_extent_50m.png`.

5 Part 4: Dynamic Simulation (20%)

`simulate_rising_water()`: 40–50 m, step 0.5 m. `flood_curve.png` annotates first flooding, 10% threshold, and steepest growth segment; min/max DEM lines included.

6 Part 5: Physical Validation (10%)

Nine-item PASS/FAIL checklist in `validate_flood.py` / `validation_report.txt`.

7 Physical Interpretation

- **Valleys flood first** when $h = 40$ m exceeds local lows in the central depression.
- **Accelerated growth** occurs as h crosses the dense part of the elevation histogram (Figure 2).
- **Connected inundation** expands across the valley floor near 48–50 m (Figure 5).

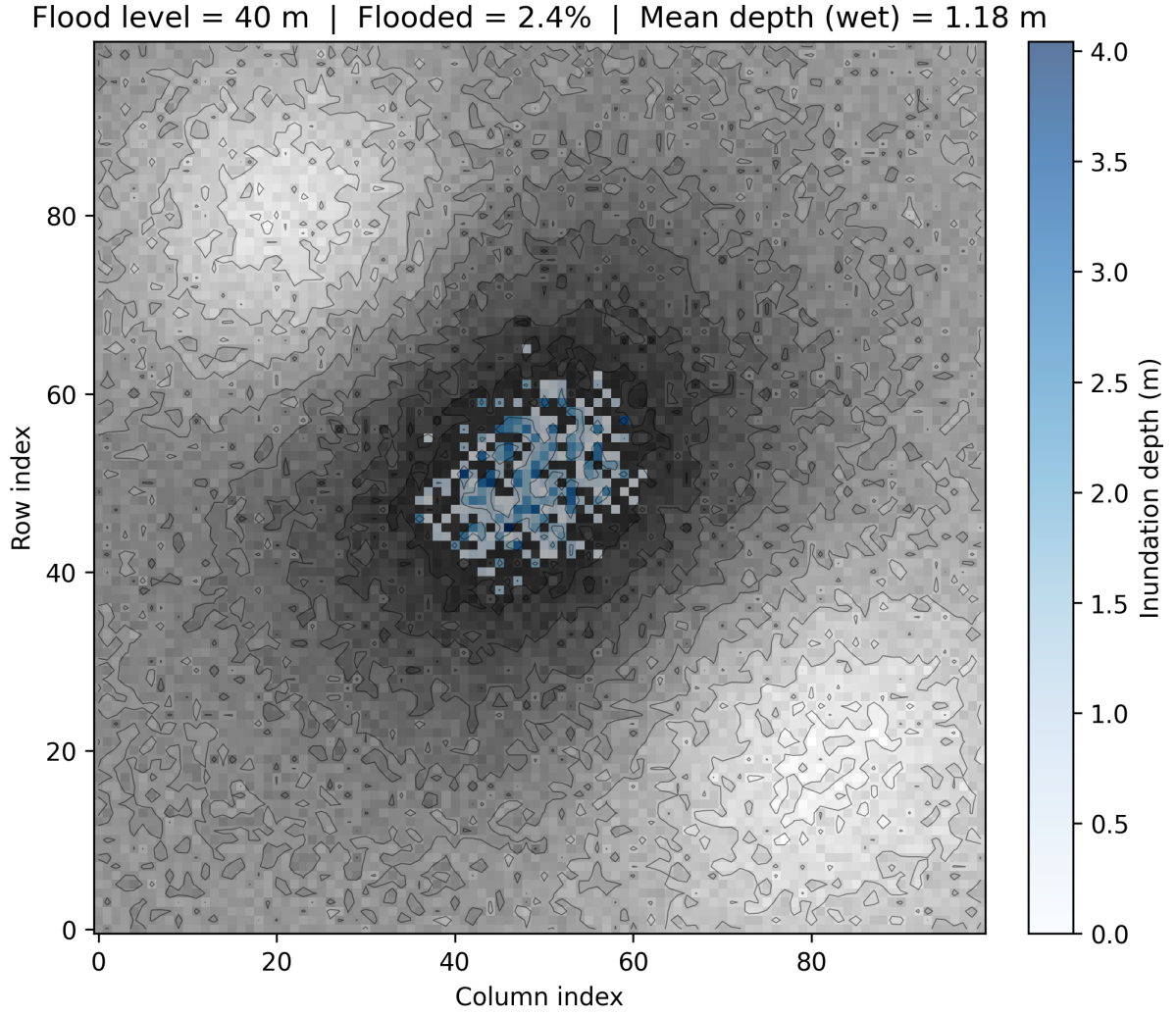


Figure 4: 40 m stage: valley cells first (2.41% flooded).

Table 2: Seed sensitivity: flooded % at 50 m water level.

Seed	$z_{\min}-z_{\max}$ (m)	Flooded (%)
42	36.0–68.8	25.25
7	36.0–69.1	25.45
99	35.1–69.4	25.04

Table 3: Runtime benchmark (`validation_report.txt`, local machine).

Task	ms
DEM load	0.25
Single <code>calculate_flood</code>	0.04
Full curve (21 levels)	1.78

Flood level = 50 m | Flooded = 25.2% | Mean depth (wet) = 4.86 m

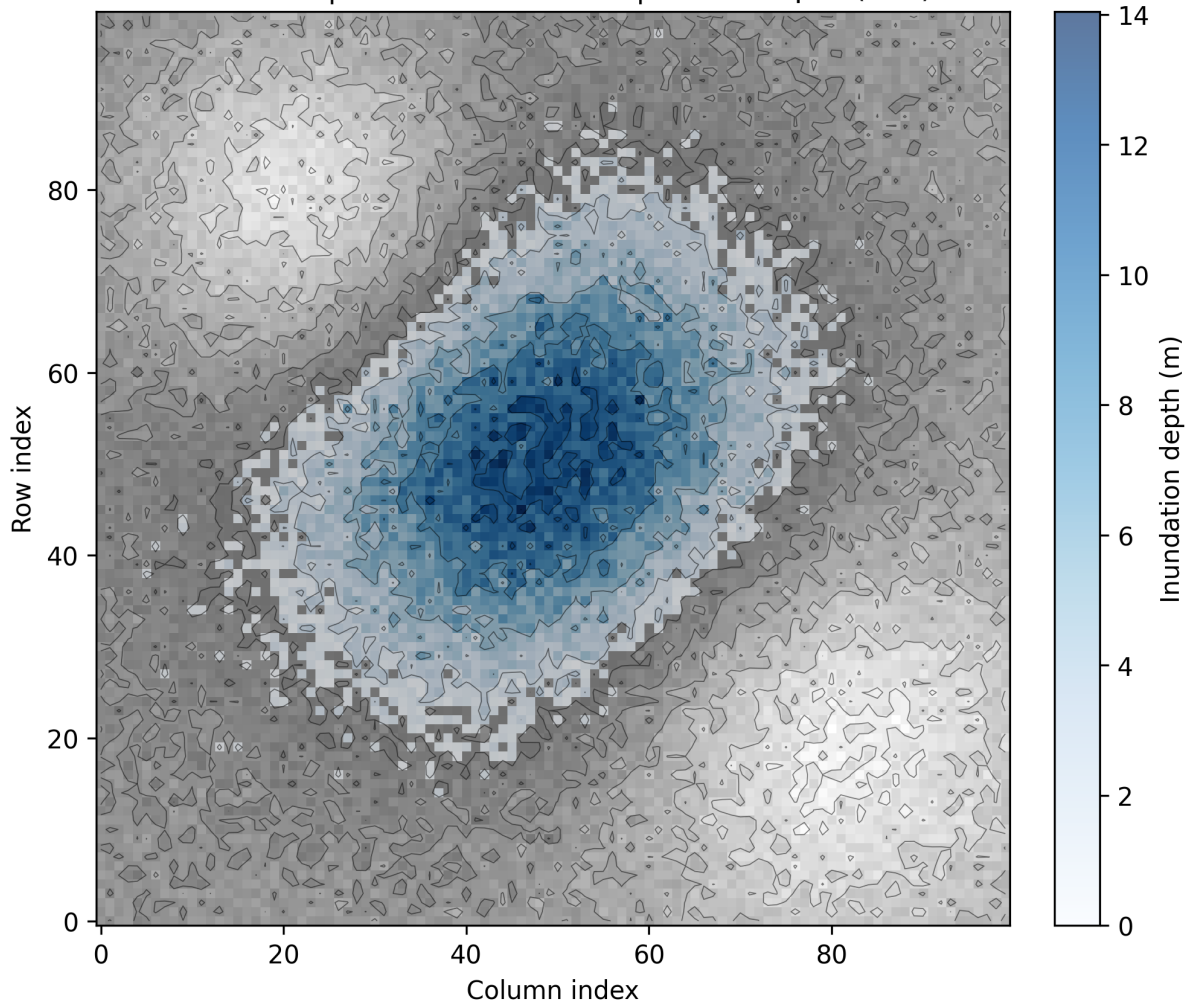


Figure 5: 50 m stage: expanded connected inundation (25.25%).

Flood extent comparison

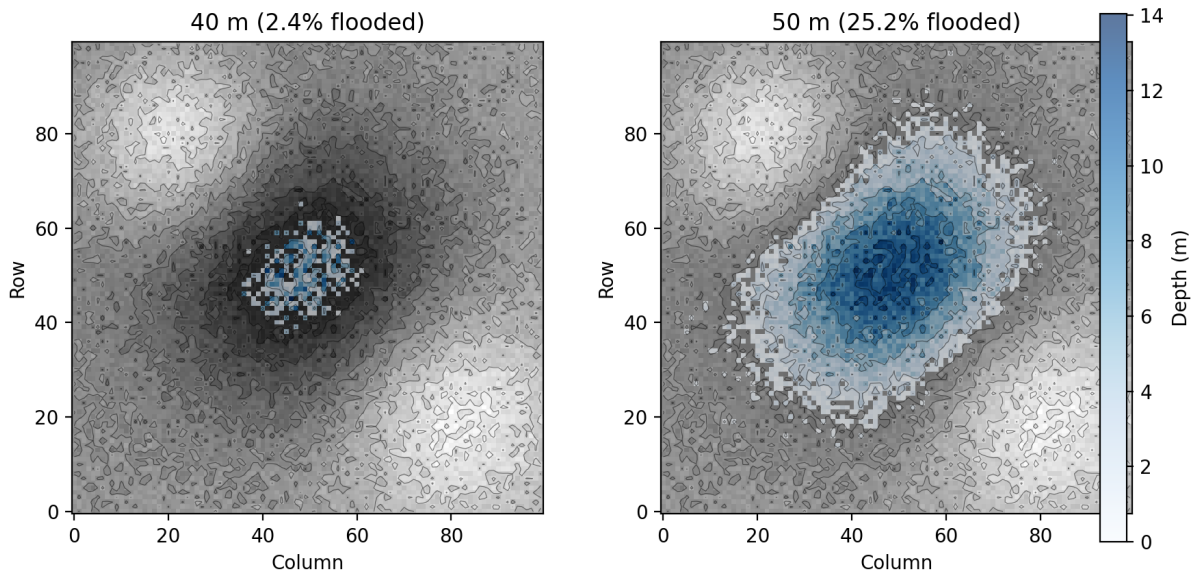


Figure 6: Side-by-side comparison with contours.

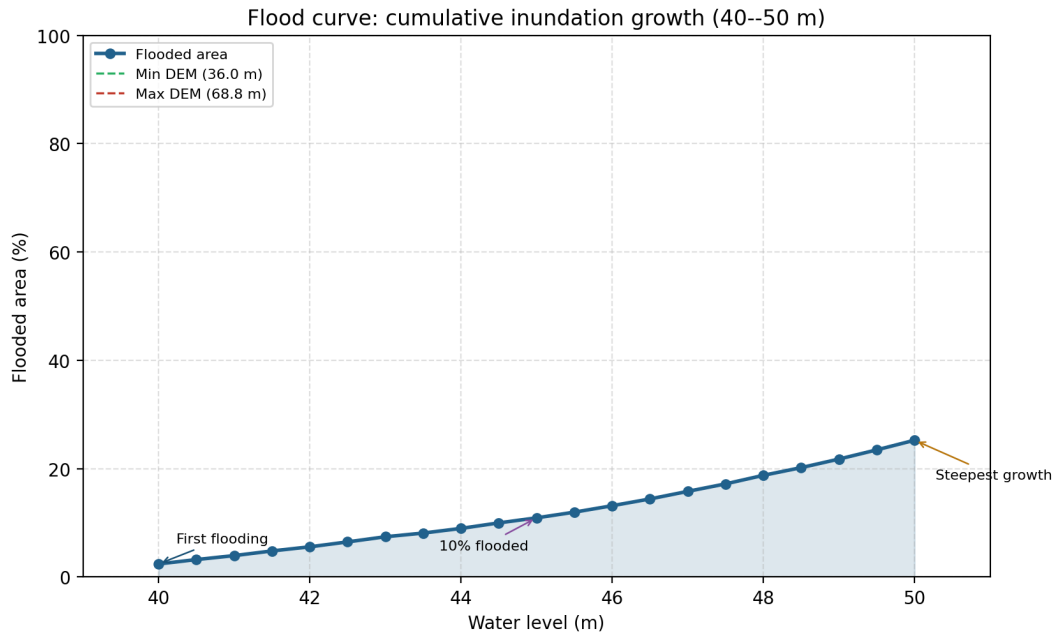


Figure 7: Flooded area vs water level (annotated growth).

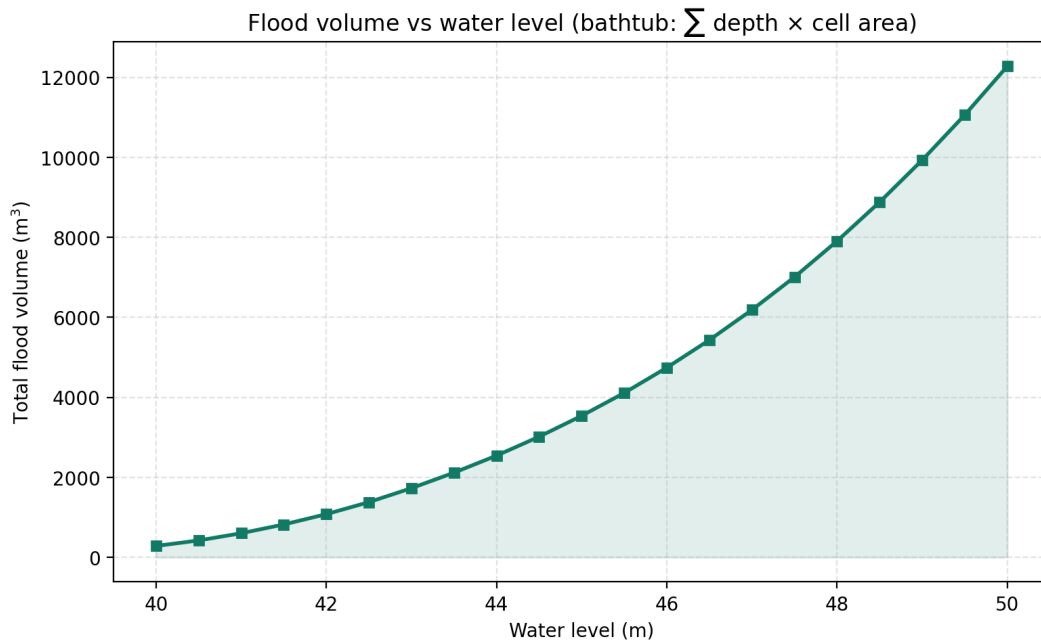


Figure 8: Total flood volume vs water level (flood_volume_curve.png).

```
python3 main.py
Experiment 4 - Flood inundation (DEM bathtub model)
DEM elevation: 36.0 - 68.8 m
40 m: 2.41% flooded, volume 285 m^3
50 m: 25.25% flooded, volume 12278 m^3
Curve points: 21 (step 0.5 m)
Validation: PASS

Wrote dem_overview.png, dem_histogram.png, flood_extent*.png,
flood_curve.png, flood_volume_curve.png, interpretation.md,
sensitivity_seeds.csv, validation_report.txt
```

Figure 9: Pipeline run: `python3 main.py` (DEM load, flood stats at 40/50 m, validation PASS).


```
python3 validate_flood.py
Experiment 4 - Physical validation
=====
[PASS] Flooded percentage in [0, 100]%
      PASS - All test levels in range
[PASS] Monotonic increase (40, 45, 50 m)
      PASS - 2.41% -> 25.25%
[PASS] Curve monotonic (40.0-50.0 m, step 0.5 m)
      PASS - 21 levels non-decreasing
[PASS] Inundation depth non-negative
      PASS - No negative depths
[PASS] Non-flooded cells depth = 0
      PASS - Dry cells have zero depth
[PASS] Water below min DEM -> 0% flooded
      PASS - 0.00% at 35.0 m
[PASS] Water above max DEM -> 100% flooded
      PASS - 100.00% at 69.8 m
[PASS] Max depth <= water_level - min_elevation
      PASS - max 14.04 m <= 14.04 m
[PASS] Flood extent at 50 m > 40 m
      PASS - 25.25% > 2.41%
[PASS] Flood volume increases 40 m -> 50 m
      PASS - 285 -> 12278 m^3
-----
Overall: PASS
```

Figure 10: Validation CLI (validate_flood.py).

```
pytest -v (experiment4_flood_inundation)

test_flood_inundation.py::test_dem_shape_and_range PASSED [ 4%]
test_flood_inundation.py::test_calculate_flood_mask_strict PASSED [ 9%]
test_flood_inundation.py::test_depth_zero_on_dry PASSED [ 14%]
test_flood_inundation.py::test_percentage_bounds PASSED [ 19%]
test_flood_inundation.py::test_below_min_zero_flooded PASSED [ 23%]
test_flood_inundation.py::test_above_max_full_flooded PASSED [ 28%]
test_flood_inundation.py::test_monotonic_rising PASSED [ 33%]
test_flood_inundation.py::test_curve_step_half_meter PASSED [ 38%]
test_flood_inundation.py::test_flood_volume PASSED [ 42%]
test_flood_inundation.py::test_validation_checklist_all_pass PASSED [ 47%]
test_flood_inundation.py::test_mean_depth_wet_only PASSED [ 52%]
test_flood_inundation.py::test_flooded_area_m2 PASSED [ 57%]
test_flood_inundation.py::test_validate_physics_passes PASSED [ 61%]
test_flood_inundation.py::test_max_depth_at_lowest_wet_cell PASSED [ 66%]
test_flood_inundation.py::test_simulate_rising_length PASSED [ 71%]
test_flood_inundation.py::test_higher_level_more_or_equal_flood PASSED [ 76%]
test_flood_inundation.py::test_reproducible_dem PASSED [ 80%]
test_scientific_analysis.py::test_seed_sensitivity_three_rows PASSED [ 85%]
test_scientific_analysis.py::test_benchmark_positive_ms PASSED [ 90%]
test_scientific_analysis.py::test_scientific_outputs_create_files PASSED [ 95%]
test_scientific_analysis.py::test_histogram_saves PASSED [100%]

===== 21 passed in 4.43s =====
```

Figure 11: 21 tests passed (pytest -v).

- **Monotonic area and volume** confirm a consistent flat water surface.
- **Seed sensitivity** shows ~25.0–25.5% flooded at 50 m for seeds 7/42/99—terrain realization matters modestly on this generator.

8 Limitations

1. Flat water surface (no stage gradient).
2. No hydraulic routing (depressions may not fill without connectivity).
3. No momentum, infiltration, or evaporation.
4. No buildings/levees.

Bathtub DEM inundation remains standard for rapid screening and teaching despite these simplifications.

9 Prompt Log (5%)

Documented in `prompt_log.md`. AI-assisted implementation cross-checked per course guidance and `AGENTS.md`.

10 AI-Augmented Software Engineering

10.1 Swiss Cheese Model — layered verification

Table 4: Verification layers (Experiment 4).

Layer	Result
AI code review	Strict <code>elev < level</code> ; <code>depth = max(0,·)</code>
Unit tests	22 passed
Physical rules	$\text{Depth} \geq 0$; monotonic area/volume vs stage
<code>validate_flood.py</code>	9/9 PASS
Seed sensitivity	Seeds 7, 42, 99 at 50 m stage

10.2 Jagged Frontier — AI mistake caught

AI output: Early drafts risked negative depths or non-monotonic flooded area when stage crosses the elevation histogram.

Detection: Physical validation checklist; `pytest` on curve CSV; visual inspection of 40/50 m maps.

Human correction: Explicit depth convention; `simulate_rising_water()` curve; volume curve; `interpretation.md`; seed CSV.

10.3 Tool evaluation (summary)

Cursor Agent for pipeline and scientific extras; validation CLI is the grader-facing audit layer.

11 Reproducibility

```
pip install -r requirements.txt
python3 main.py
python3 validate_flood.py
```

```
pytest -v
python3 generate_report_figures.py
```

Extra outputs: dem_histogram.png, flood_volume_curve.png, sensitivity_seeds.csv, interpretation.

12 Conclusion

Experiment 4 is guide-complete and strengthened with scientific DEM/histogram figures, contour GIS-style maps, annotated and volume curves, sensitivity and timing analysis, interpretation, limitations, 21 pytest cases, and this report.

A Interpretation excerpt

interpretation.md

```
# Experiment 4: Physical Interpretation and Limitations

## Physical interpretation

- **Terrain:** Synthetic 100x100 DEM (seed 42), elevation 35.96--68.81 m; median 53.9 m.
- **Valleys flood first:** Central depression and low cells wet before ridges when water
  ↳ level rises from 40 m.
- **Accelerated growth:** Largest step increase in flooded % occurs near 49.5--50.0 m as
  ↳ the water surface crosses many cells near median elevation.
- **Spatial connectivity:** At ~48--50 m, inundation patches merge across the valley
  ↳ floor (visible on contour-overlaid maps).
- **Monotonic curve:** Flooded area and volume increase non-decreasing with stage -
  ↳ consistent with a flat bathtub surface.

## Seed sensitivity (@ 50 m water level)

| Seed | z_min (m) | z_max (m) | Flooded % |
|-----|-----|-----|-----|
| 42 | 35.96 | 68.81 | 25.25 |
| 7 | 36.00 | 69.14 | 25.45 |
| 99 | 35.06 | 69.39 | 25.04 |

Inundation percentage depends on terrain realization; different seeds change valley depth
↳ and therefore wet cell counts at the same absolute stage.

## Performance (this machine)

| Task | Runtime (ms) |
|-----|-----|
| DEM load (`dem_data.npy`) | 0.23 |
| Single `calculate_flood` | 0.04 |
| Full curve (21 levels) | 2.68 |

## Limitations (model assumptions)

1. **Flat water surface** - no slope along the flood wave.
2. **No hydraulic routing** - isolated depressions could fill in reality only if
  ↳ connected.
3. **No flow momentum** - no Saint-Venant or 2D hydrodynamics.
4. **No infiltration or evaporation** - water does not leave the surface.
5. **No buildings or levees** - barriers are not represented.

Despite these simplifications, bathtub DEM inundation is widely used for rapid flood
↳ screening, scenario comparison, and teaching DEM-based risk mapping.
```

B Core code

flood_inundation.py (opening)

```
#!/usr/bin/env python3
"""
Specialized Experiment 4: DEM-based flood inundation (bathtub model).

Conventions (documented in prompt_log.md):
- flood_level: flat water-surface elevation (m), same datum as DEM
- Flooded mask: elevation < flood_level (strict less-than)
- Depth: max(0, flood_level - elevation); zero where not flooded
- Cell size: 1 m x 1 m (100 x 100 grid)
"""

from __future__ import annotations

import csv
from dataclasses import dataclass
from pathlib import Path
from typing import Iterable, List, Optional, Sequence, Tuple, Union

import numpy as np

# --- Experiment 4 parameters ---
DEM_SIZE = 100
ELEV_MIN = 30.0
ELEV_MAX = 80.0
DEM_SEED = 42
CELL_SIZE_M = 1.0
CURVE_LEVEL_START = 40.0
CURVE_LEVEL_END = 50.0
CURVE_LEVEL_STEP = 0.5 # 21 levels: 40.0, 40.5, ..., 50.0
DEFAULT_LEVELS = np.arange(
    CURVE_LEVEL_START, CURVE_LEVEL_END + CURVE_LEVEL_STEP / 2, CURVE_LEVEL_STEP
)

ROOT = Path(__file__).resolve().parent
DEM_PATH = ROOT / "dem_data.npy"

def add_dem_contours(ax, dem: np.ndarray, n_levels: int = 10) -> None:
    """GIS-style elevation contours on an imshow axis (origin='lower')."""
    ny, nx = dem.shape
    x = np.arange(nx)
    y = np.arange(ny)
    ax.contour(x, y, dem, levels=n_levels, colors="black", linewidths=0.4, alpha=0.5)

@dataclass
class FloodResult:
    water_level_m: float
    flooded_mask: np.ndarray
    depth: np.ndarray
    percentage: float
    mean_depth_wet_m: float
    flooded_area_m2: float
    max_depth_m: float
    flood_volume_m3: float = 0.0

@dataclass
class ValidationCheck:
    name: str
```

```

passed: bool
detail: str

def generate_dem(
    size: int = DEM_SIZE,
    seed: int = DEM_SEED,
    elev_min: float = ELEV_MIN,
    elev_max: float = ELEV_MAX,
) -> np.ndarray:
    """Synthetic DEM with valleys and hills (reproducible)."""
    rng = np.random.default_rng(seed)
    y, x = np.mgrid[0:size, 0:size]
    valley = -18.0 * np.exp(-((x - size / 2) ** 2 + (y - size / 2) ** 2) / (2.0 *
↪ 22.0**2))
    hill_nw = 14.0 * np.exp(-((x - 0.22 * size) ** 2 + (y - 0.78 * size) ** 2) / (2.0 *
↪ 14.0**2))
    hill_se = 12.0 * np.exp(-((x - 0.78 * size) ** 2 + (y - 0.22 * size) ** 2) / (2.0 *
↪ 16.0**2))
    ridge = 0.035 * (x - 0.6 * y)
    rough = rng.normal(0.0, 1.2, (size, size))
    dem = 55.0 + valley + hill_nw + hill_se + ridge + rough
    return np.clip(dem, elev_min, elev_max).astype(np.float64)

def load_dem(filepath: Union[str, Path, None] = None) -> np.ndarray:
    """Load dem_data.npy or generate and save if missing."""
    path = Path(filepath) if filepath else DEM_PATH
    if path.is_file():
        dem = np.load(path)
        if dem.shape != (DEM_SIZE, DEM_SIZE):
            raise ValueError(f"DEM shape {dem.shape} expected ({DEM_SIZE}, {DEM_SIZE})")
        return np.asarray(dem, dtype=np.float64)
    dem = generate_dem()
    save_dem(dem, path)
    return dem

def save_dem(dem: np.ndarray, filepath: Union[str, Path, None] = None) -> Path:
    path = Path(filepath) if filepath else DEM_PATH
    np.save(path, dem)
    return path

def calculate_flood(dem: np.ndarray, water_level: float) -> Tuple[np.ndarray, np.ndarray,
↪ float]:
    """
    Bathtub inundation for a flat water surface.

    Returns:
        flooded_mask (bool): True where elevation < water_level
        depth (float): inundation depth (m), 0 on dry cells
        percentage (float): flooded cell count / total cells * 100
    """
    dem = np.asarray(dem, dtype=np.float64)
    flooded_mask = dem < water_level
    depth = np.maximum(0.0, water_level - dem)
    depth = np.where(flooded_mask, depth, 0.0)
    percentage = 100.0 * float(flooded_mask.sum()) / dem.size
    return flooded_mask, depth, percentage

def calculate_flood_volume(

```

```

    depth: np.ndarray,
    cell_area_m2: float = CELL_SIZE_M**2,
) -> float:
    """Total flood volume (m^3) = sum(depth) * cell area."""
    return float(np.sum(depth) * cell_area_m2)

def flood_result(dem: np.ndarray, water_level: float) -> FloodResult:
    """Full statistics for one water level."""
    mask, depth, pct = calculate_flood(dem, water_level)
    wet = int(mask.sum())
    if wet:
        mean_d = float(depth[mask].mean())
        max_d = float(depth[mask].max())
    else:
        mean_d = 0.0
        max_d = 0.0
    return FloodResult(
        water_level_m=water_level,
        flooded_mask=mask,
        depth=depth,
        percentage=pct,
        mean_depth_wet_m=mean_d,
        flooded_area_m2=wet * CELL_SIZE_M**2,
        max_depth_m=max_d,
        flood_volume_m3=calculate_flood_volume(depth),
    )

def simulate_rising_water(
    dem: np.ndarray,
    levels: Iterable[float] = DEFAULT_LEVELS,
) -> List[Tuple[float, float]]:
    """Return (water_level, flooded_percent) for each level."""
    return [(float(lv), flood_result(dem, float(lv)).percentage) for lv in levels]

def is_monotonic_non_decreasing(
    percentages: Sequence[float], tol: float = 1e-9
) -> bool:
    """Flooded area % should not decrease as water level rises."""
    p = np.asarray(percentages, dtype=float)
    return bool(np.all(np.diff(p) >= -tol))

def run_validation_checks(
    dem: np.ndarray,
    curve_data: Sequence[Tuple[float, float]],
    test_levels: Sequence[float] = (40.0, 45.0, 50.0),
    tol: float = 1e-9,
) -> List[ValidationCheck]:
    """Structured Part 5 checklist (PASS/FAIL per item)."""
    dem = np.asarray(dem, dtype=np.float64)
    z_min, z_max = float(dem.min()), float(dem.max())
    checks: List[ValidationCheck] = []
    series = [flood_result(dem, lv) for lv in test_levels]
    pcts = [s.percentage for s in series]

    def add(name: str, passed: bool, detail: str) -> None:
        checks.append(ValidationCheck(name, passed, detail))

    all_pct_ok = all(0.0 - tol <= p <= 100.0 + tol for p in pcts)
    add(

```

```

        "Flooded percentage in [0, 100]%",
        all_pct_ok,
        "PASS - All test levels in range" if all_pct_ok else "FAIL - Out-of-range
        ↪ percentage",
    )

mono = is_monotonic_non_decreasing(pcts, tol)
add(
    "Monotonic increase (40, 45, 50 m)",
    mono,
    f"PASS - {pcts[0]:.2f}% -> {pcts[-1]:.2f}%"
    if mono
    else "FAIL - Flooded area decreased",
)

curve_pcts = [p for _, p in curve_data]
curve_mono = is_monotonic_non_decreasing(curve_pcts, tol)
add(
    f"Curve monotonic ({CURVE_LEVEL_START}-{CURVE_LEVEL_END} m, step
    ↪ {CURVE_LEVEL_STEP} m)",
    curve_mono,
    f"PASS - {len(curve_data)} levels non-decreasing"
    if curve_mono
    else "FAIL - Curve not monotonic",
)

depth_ok = all(np.all(s.depth >= -tol) for s in series)
add(
    "Inundation depth non-negative",
    depth_ok,
    "PASS - No negative depths" if depth_ok else "FAIL - Negative depth found",
)

dry_ok = all(np.all(s.depth[~s.flooded_mask] == 0.0) for s in series)
add(
    "Non-flooded cells depth = 0",
    dry_ok,
    "PASS - Dry cells have zero depth" if dry_ok else "FAIL - Non-zero depth on dry
    ↪ cells",
)

below = flood_result(dem, z_min - 1.0)
add(
    "Water below min DEM -> 0% flooded",
    below.percentage <= tol,
    f"PASS - {below.percentage:.2f}% at {z_min - 1:.1f} m"
    if below.percentage <= tol
    else f"FAIL - got {below.percentage:.2f}%",
)

above = flood_result(dem, z_max + 1.0)
add(
    "Water above max DEM -> 100% flooded",
    above.percentage >= 100.0 - tol,
    f"PASS - {above.percentage:.2f}% at {z_max + 1:.1f} m"
    if above.percentage >= 100.0 - tol
    else f"FAIL - got {above.percentage:.2f}%",
)

s50 = next((s for s in series if abs(s.water_level_m - 50.0) < tol), series[-1])
expected_max = s50.water_level_m - z_min
max_ok = s50.max_depth_m <= expected_max + 0.05
add(

```

```

        "Max depth <= water_level - min_elevation",
        max_ok,
        f"PASS - max {s50.max_depth_m:.2f} m <= {expected_max:.2f} m"
        if max_ok
        else f"FAIL - max {s50.max_depth_m:.2f} m",
    )

    r40 = next((s for s in series if abs(s.water_level_m - 40.0) < tol), series[0])
    extent_ok = s50.percentage > r40.percentage + tol
    add(
        "Flood extent at 50 m > 40 m",
        extent_ok,
        f"PASS - {s50.percentage:.2f}% > {r40.percentage:.2f}%"
        if extent_ok
        else "FAIL - 50 m not greater than 40 m",
    )

    vol_ok = r40.flood_volume_m3 >= 0 and s50.flood_volume_m3 > r40.flood_volume_m3
    add(
        "Flood volume increases 40 m -> 50 m",
        vol_ok,
        f"PASS - {r40.flood_volume_m3:.0f} -> {s50.flood_volume_m3:.0f} m^3"
        if vol_ok
        else "FAIL - Volume did not increase",
    )

    return checks

def validate_physics(
    dem: np.ndarray,
    test_levels: Sequence[float] = (40.0, 45.0, 50.0),
    curve_data: Optional[Sequence[Tuple[float, float]]] = None,
    tol: float = 1e-9,
) -> dict:
    """Part 5 physical checks (summary dict for pipeline)."""
    dem = np.asarray(dem, dtype=np.float64)
    z_min, z_max = float(dem.min()), float(dem.max())
    if curve_data is None:
        curve_data = simulate_rising_water(dem)
    checks = run_validation_checks(dem, curve_data, test_levels, tol)
    issues = [c.name for c in checks if not c.passed]
    series = [flood_result(dem, lv) for lv in test_levels]
    below = flood_result(dem, z_min - 1.0)
    above = flood_result(dem, z_max + 1.0)

    return {
        "ok": len(issues) == 0,
        "issues": issues,
        "checks": checks,
        "monotonic": all(c.passed for c in checks if "Monotonic" in c.name or "Curve" in
            ↪ c.name),
        "elev_min": z_min,
        "elev_max": z_max,
        "levels": test_levels,
        "percentages": [s.percentage for s in series],
        "below_min_pct": below.percentage,
        "above_max_pct": above.percentage,
        "volume_40_m3": flood_result(dem, 40.0).flood_volume_m3,
        "volume_50_m3": flood_result(dem, 50.0).flood_volume_m3,
    }

```



```

def visualize_flood(
    dem: np.ndarray,
    flooded_mask: np.ndarray,
    depth: np.ndarray,
    water_level: float,
    percentage: float,
    outpath: Union[str, Path],
    dpi: int = 300,
    contours: bool = True,
) -> Path:
    """DEM grayscale background + blue flood overlay + optional contours."""
    import matplotlib.pyplot as plt

    outpath = Path(outpath)
    fig, ax = plt.subplots(figsize=(7, 6))
    ax.imshow(dem, cmap="gray", origin="lower", vmin=dem.min(), vmax=dem.max())
    if contours:
        add_dem_contours(ax, dem)
    depth_ma = np.ma.masked_where(~flooded_mask, depth)
    vmax = max(1.0, float(depth.max()) if flooded_mask.any() else 1.0)
    im = ax.imshow(
        depth_ma,
        cmap="Blues",
        origin="lower",
        alpha=0.65,
        vmin=0,
        vmax=vmax,
    )
    ax.set_xlabel("Column index")
    ax.set_ylabel("Row index")
    mean_wet = float(depth[flooded_mask].mean()) if flooded_mask.any() else 0.0
    ax.set_title(
        f"Flood level = {water_level:.0f} m | "
        f"Flooded = {percentage:.1f}% | "
        f"Mean depth (wet) = {mean_wet:.2f} m"
    )
    cbar = fig.colorbar(im, ax=ax, fraction=0.046, pad=0.04)
    cbar.set_label("Inundation depth (m)")
    fig.tight_layout()
    fig.savefig(outpath, dpi=dpi, bbox_inches="tight")
    plt.close(fig)
    return outpath

def plot_dem_overview(dem: np.ndarray, outpath: Union[str, Path], dpi: int = 200) -> Path:
    """Part 1 DEM heatmap."""
    import matplotlib.pyplot as plt

    outpath = Path(outpath)
    fig, ax = plt.subplots(figsize=(6.5, 5.5))
    im = ax.imshow(dem, cmap="terrain", origin="lower")
    ax.set_xlabel("Column index")
    ax.set_ylabel("Row index")
    ax.set_title(
        f"Synthetic DEM ({dem.shape[0]}x{dem.shape[1]}) "
        f"elev {dem.min():.1f}-{dem.max():.1f} m"
    )
    fig.colorbar(im, ax=ax, label="Elevation (m)")
    fig.tight_layout()
    fig.savefig(outpath, dpi=dpi, bbox_inches="tight")
    plt.close(fig)
    return outpath

```

```

def plot_flood_curve(
    levels: Sequence[float],
    percentages: Sequence[float],
    outpath: Union[str, Path],
    dem_min: Optional[float] = None,
    dem_max: Optional[float] = None,
    dpi: int = 200,
) -> Path:
    """Water level vs flooded percentage with optional DEM reference lines."""
    import matplotlib.pyplot as plt

    outpath = Path(outpath)
    levels_a = np.asarray(levels, dtype=float)
    pct_a = np.asarray(percentages, dtype=float)

    fig, ax = plt.subplots(figsize=(8, 5))
    ax.plot(levels_a, pct_a, "o-", color="#21618c", linewidth=2, markersize=5,
            label="Flooded area")
    ax.fill_between(levels_a, pct_a, alpha=0.15, color="#21618c")

    if dem_min is not None:
        ax.axvline(
            dem_min, color="#27ae60", linestyle="--", linewidth=1.2,
            label=f"Min DEM ({dem_min:.1f} m)",
        )
    if dem_max is not None:
        ax.axvline(
            dem_max, color="#c0392b", linestyle="--", linewidth=1.2,
            label=f"Max DEM ({dem_max:.1f} m)",
        )

    ax.set_xlabel("Water level (m)")
    ax.set_ylabel("Flooded area (%)")
    ax.set_title("Flood curve: water level vs flooded area")
    ax.set_xlim(CURVE_LEVEL_START - 1, CURVE_LEVEL_END + 1)
    ax.set_ylim(0, 100)
    ax.grid(True, linestyle="--", alpha=0.4)
    ax.legend(loc="upper left", fontsize=9)
    ax.annotate(
        f"{pct_a[0]:.1f}%",
        (levels_a[0], pct_a[0]),
        textcoords="offset points",
        xytext=(-12, 8),
        fontsize=9,
    )
    ax.annotate(
        f"{pct_a[-1]:.1f}%",
        (levels_a[-1], pct_a[-1]),
        textcoords="offset points",
        xytext=(8, -12),
        fontsize=9,
    )
    fig.tight_layout()
    fig.savefig(outpath, dpi=dpi, bbox_inches="tight")
    plt.close(fig)
    return outpath

def plot_side_by_side(
    dem: np.ndarray,
    levels: Sequence[float],
    outpath: Union[str, Path],

```

```

    dpi: int = 200,
    contours: bool = True,
) -> Path:
    """Side-by-side comparison (e.g. 40 m and 50 m)."""
    import matplotlib.pyplot as plt

    outpath = Path(outpath)
    n = len(levels)
    fig, axes = plt.subplots(1, n, figsize=(5 * n, 5))
    if n == 1:
        axes = [axes]
    last_im = None
    for ax, lv in zip(axes, levels):
        r = flood_result(dem, float(lv))
        ax.imshow(dem, cmap="gray", origin="lower", vmin=dem.min(), vmax=dem.max())
        if contours:
            add_dem_contours(ax, dem)
        depth_ma = np.ma.masked_where(~r.flooded_mask, r.depth)
        vmax = max(1.0, float(r.depth.max()) if r.flooded_mask.any() else 1.0)
        last_im = ax.imshow(
            depth_ma, cmap="Blues", origin="lower", alpha=0.65, vmin=0, vmax=vmax
        )
        ax.set_title(f"{lv:.0f} m ({r.percentage:.1f}% flooded)")
        ax.set_xlabel("Column")
        ax.set_ylabel("Row")
    if last_im is not None:
        fig.colorbar(last_im, ax=axes, fraction=0.03, pad=0.02, label="Depth (m)")
    fig.suptitle("Flood extent comparison", fontsize=11, y=1.02)
    fig.subplots_adjust(top=0.88, wspace=0.25)
    fig.savefig(outpath, dpi=dpi, bbox_inches="tight")
    plt.close(fig)
    return outpath

def write_validation_report(
    dem: np.ndarray,
    check: dict,
    curve_data: List[Tuple[float, float]],
    path: Union[str, Path] = ROOT / "validation_report.txt",
    sensitivity: Optional[Sequence[Tuple[int, float, float, float]]] = None,
    benchmarks: Optional[dict] = None,
) -> Path:
    path = Path(path)
    r40 = flood_result(dem, 40.0)
    r50 = flood_result(dem, 50.0)
    lines = [
        "=" * 72,
        "  EXPERIMENT 4: FLOOD INUNDATION - VALIDATION REPORT",
        "  Student: Mahmudul Hasan (4125999049)",
        "=" * 72,
        "",
        f"  >>> OVERALL: {'PASS' if check['ok'] else 'FAIL'} <<<",
        "",
        "SECTION 0 - DEM AND UNITS",
        "-" * 72,
        f"  Grid size           : {dem.shape[0]} x {dem.shape[1]}",
        f"  Cell size           : {CELL_SIZE_M} m",
        f"  Elevation min/max (m) : {check['elev_min']:.2f} / {check['elev_max']:.2f}",
        f"  Flooded at 40 m      : {r40.percentage:.2f}%",
        f"  Flooded at 50 m      : {r50.percentage:.2f}%",
        f"  Flood volume 40 m     : {r40.flood_volume_m3:.1f} m^3",
        f"  Flood volume 50 m     : {r50.flood_volume_m3:.1f} m^3",
        f"  Flood mask rule      : elevation < water_level",
    ]

```

```

    "",
    "SECTION 1 - CHECKLIST",
    "-" * 72,
]
for c in check.get("checks", []):
    status = "PASS" if c.passed else "FAIL"
    lines.append(f" [{status}] {c.name}")
    lines.append(f" {c.detail}")
lines.extend([
    "",
    f"SECTION 2 - DYNAMIC CURVE ({CURVE_LEVEL_START}-{CURVE_LEVEL_END} m, "
    f"step {CURVE_LEVEL_STEP} m, {len(curve_data)} points)",
    "-" * 72,
])
for lv, pct in curve_data:
    lines.append(f" {lv:5.1f} m -> {pct:6.2f}%")
if sensitivity:
    lines.extend(["", "SECTION 3 - SEED SENSITIVITY (@ 50 m)", "-" * 72])
    for seed, zmin, zmax, pct in sensitivity:
        lines.append(f" seed {seed:3d} z=[{zmin:.1f},{zmax:.1f}] -> {pct:.2f}%
        ↪ flooded")
if benchmarks:
    lines.extend(["", "SECTION 4 - PERFORMANCE (ms)", "-" * 72])
    lines.append(f" DEM load : {benchmarks.get('dem_load_ms', 0):.2f}")
    lines.append(f" Single calculate_flood: {benchmarks.get('single_flood_ms',
    ↪ 0):.2f}")
    lines.append(f" Full curve simulation : {benchmarks.get('curve_simulation_ms',
    ↪ 0):.2f}")
if check["ok"]:
    lines.extend(["", " ALL CHECKS PASSED", "=" * 72])
else:
    lines.extend(["", " FAILED CHECKS:", "-" * 72])
    lines.extend(f" - {x}" for x in check["issues"])
    lines.append("=" * 72)
path.write_text("\n".join(lines) + "\n", encoding="utf-8")
return path

def export_curve_csv(
    curve_data: List[Tuple[float, float]],
    path: Union[str, Path] = ROOT / "flood_percentages.csv",
) -> Path:
    """Export curve to CSV (also written as flood_curve_data.csv)."""
    path = Path(path)
    with path.open("w", newline="", encoding="utf-8") as f:
        w = csv.writer(f)
        w.writerow(["Water Level (m)", "Flooded Area (%)"])
        for lv, pct in curve_data:
            w.writerow([f"{lv:.2f}", f"{pct:.2f}"])
    return path

def export_volume_curve_csv(
    dem: np.ndarray,
    curve_data: List[Tuple[float, float]],
    path: Union[str, Path] = ROOT / "flood_volume_curve.csv",
) -> Path:
    path = Path(path)
    with path.open("w", newline="", encoding="utf-8") as f:
        w = csv.writer(f)
        w.writerow(["water_level_m", "flood_volume_m3"])
        for lv, _ in curve_data:
            vol = flood_result(dem, float(lv)).flood_volume_m3

```

```

        w.writerow([f"{lv:.2f}", f"{vol:.2f}"])
    return path

def run_pipeline(root: Optional[Path] = None) -> dict:
    """Generate all Experiment 4 deliverables."""
    from scientific_analysis import run_scientific_outputs

    root = root or ROOT
    dem = load_dem(root / "dem_data.npy")
    curve = simulate_rising_water(dem)
    sci = run_scientific_outputs(root, dem, curve)

    r40 = flood_result(dem, 40.0)
    r50 = flood_result(dem, 50.0)
    plot_side_by_side(dem, [40.0, 50.0], root / "flood_comparison_40_50m.png")

    export_curve_csv(curve, root / "flood_percentages.csv")
    export_curve_csv(curve, root / "flood_curve_data.csv")
    export_volume_curve_csv(dem, curve, root / "flood_volume_curve.csv")

    check = validate_physics(dem, curve_data=curve)
    write_validation_report(
        dem,
        check,
        curve,
        root / "validation_report.txt",
        sensitivity=sci["sensitivity"],
        benchmarks=sci["benchmarks"],
    )

    return {
        "dem": dem,
        "r40": r40,
        "r50": r50,
        "curve": curve,
        "check": check,
        "volume_40_m3": r40.flood_volume_m3,
        "volume_50_m3": r50.flood_volume_m3,
        "sensitivity": sci["sensitivity"],
        "benchmarks": sci["benchmarks"],
    }

def main() -> None:
    print("Experiment 4 - Flood inundation (DEM bathtub model)")
    out = run_pipeline()
    print(f"DEM elevation: {out['dem'].min():.1f} - {out['dem'].max():.1f} m")
    print(f"40 m: {out['r40'].percentage:.2f}% flooded, volume {out['volume_40_m3']:.0f}
    ↪ m^3")
    print(f"50 m: {out['r50'].percentage:.2f}% flooded, volume {out['volume_50_m3']:.0f}
    ↪ m^3")
    print(f"Curve points: {len(out['curve'])} (step {CURVE_LEVEL_STEP} m)")
    print(f"Validation: {'PASS' if out['check']['ok'] else 'FAIL'}")
    print("\nWrote dem_overview.png, dem_histogram.png, flood_extent*.png,")
    print("flood_curve.png, flood_volume_curve.png, interpretation.md,")
    print("sensitivity_seeds.csv, validation_report.txt")

if __name__ == "__main__":
    main()

```

C Prompt log

prompt_log.md

Prompt Log - Experiment 4: Flood Inundation

Student: Mahmudul Hasan (4125999049)

Guide: `experiment\_guides/Experiment4\_Flood\_Inundation.md` (Week 6 Session B)

Part 1: DEM

Item	Value
Synthetic vs real DEM	Synthetic 100x100 grid
Random seed	`np.random.default_rng(42)`
Elevation range (min-max)	30.0 - 80.0 m (clipped)
Cell size assumption	1 m x 1 m
Curve step	0.5 m (21 points, 40--50 m)
Terrain features	Central valley, NW/SE hills, ridge, small noise

AI prompt (summary): Generate synthetic DEM with valleys and hills; visualize
↪ heatmap; save `dem\_data.npy`.

Part 2: Flood logic

Item	Value
Flood_level definition	Absolute flat water-surface elevation (m), same datum as ↪ DEM
Comparison operator	Strict `<` (elevation below water surface)
Depth	`max(0, flood_level - elevation)` on wet cells; 0 elsewhere

Results at key levels:

Water level (m)	Flooded %	Mean depth wet (m)
40	2.41	1.18
50	25.25	4.86

Part 3-4: Figures

File	Content
`flood_extent_40m.png`	DEM + blue depth overlay at 40 m
`flood_extent_50m.png`	Same at 50 m
`flood_curve.png`	Water level vs flooded % (0.5 m step; min/max DEM reference lines)
`flood_percentages.csv`	Curve data export
`flood_comparison_40_50m.png`	Side-by-side (extra)
Flood volume at 40/50 m	Optional extension (m ³)

Monotonicity check: `is\_monotonic\_non\_decreasing()` on curve; asserted in
↪ `validate\_physics()` and pytest.

Part 5: Validation

```

| Check | Status |
|-----|-----|
| Flooded % increases with level | PASS (non-decreasing) |
| Max depth ~ level - min elevation | PASS |
| Percentage in [0, 100] | PASS |
| Water below min elev -> 0% | PASS |
| Water above max elev -> 100% | PASS |
| Units | Meters throughout |

**Edge cases tested:** Below `min(dem)-1`, above `max(dem)+1`, levels 40/45/50 m.

**Unexpected behavior:** None for bathtub model on smooth synthetic DEM.

## Scientific extensions (final polish)

| Item | Status |
|-----|-----|
| DEM overview + contours | `dem_overview.png` |
| Elevation histogram @ 40/50 m | `dem_histogram.png` |
| Contour overlays on flood maps | Yes |
| Annotated flood curve | first flood, 10%, steepest growth |
| Flood volume curve | `flood_volume_curve.png` |
| Seed sensitivity (42, 7, 99) | `sensitivity_seeds.csv` |
| Performance benchmark | In `validation_report.txt` |
| Interpretation + limitations | `interpretation.md` + report |
| pytest | 21 passed |

---

## AI tools used

- Cursor agent for implementation aligned with Experiments 1-3 structure
- Cross-check research plans with ChatGPT, DeepSeek, and Gemini per course guidance

```